Pusher Protocol — FAQ

Real-time WebSocket infrastructure: concepts, history, and ecosystem

1. Who invented the Pusher protocol?

The Pusher protocol was invented by the team at **Pusher Ltd**, a British startup founded by Max Williams, Damien Tanner, and James Fisher. They built it as the communication layer for their cloud real-time messaging service at pusher.com. The protocol became so widely adopted that it outgrew the company and is now treated as an open standard independently implemented by many projects.

2. Does Laravel Reverb use the Pusher protocol?

Yes. Laravel Reverb is a self-hosted WebSocket server that fully implements the Pusher protocol. This means any Pusher-compatible client library — including `dart_pusher_channels` for Flutter, `laravel-echo` for JavaScript, and the official Pusher SDKs — works with Reverb without any modification. You get the full protocol (channels, auth, presence, events) running on your own server.

```
pusher.com           ← original paid cloud service
Laravel Reverb       ← self-hosted, same protocol, you own the server
dart_pusher_channels ← Flutter client, speaks Pusher protocol
```

3. When did Pusher appear?

Pusher was founded in **2010** in London. It launched publicly and gained rapid adoption among web developers looking for a simple hosted solution for real-time features without managing WebSocket infrastructure themselves.

4. How did people handle real-time before Pusher?

Before Pusher (and before WebSockets were stable), developers used several workarounds:

- **Short polling** — the client repeatedly sends HTTP requests every few seconds asking "anything new?" Very wasteful.

- **Long polling** — the server holds the request open until there is new data, then responds. The client immediately opens a new request. Less wasteful but still HTTP overhead on every message.
- **Server-Sent Events (SSE)** — a one-way stream from server to client over HTTP. Good for notifications but no bidirectional communication.
- **Flash/Java sockets** — browser plugins (Adobe Flash, Java Applets) had real socket support. Developers used these for games and chat before WebSockets existed.
- **iframe tricks** — a hidden iframe with a streaming response was used to push data from server to client.

WebSockets were standardised in 2011 (RFC 6455), but browser support was inconsistent for years. Pusher abstracted all of this away — it handled fallbacks, reconnections, and cross-browser compatibility so developers could just subscribe to a channel and receive events.

5. Is Pusher the only option? What are the alternatives?

No — there is a rich ecosystem. Here is a detailed breakdown:

Service / Library	Type	Protocol	Notes
pusher.com	Paid cloud	Pusher	The original. Easy to start, scales well, but costs money at scale.
Ably	Paid cloud	Own + Pusher-compatible	Enterprise-grade, global edge network, 99.999% SLA. More powerful than Pusher but more complex. Has its own SDK and also supports the Pusher protocol.
Firebase Realtime DB / Firestore	Paid cloud (Google)	Own (WebSocket)	Not a messaging protocol — it is a real-time database. Clients subscribe to data paths and receive updates automatically. Very easy for simple apps, but you are locked into Google's ecosystem.
PubNub	Paid cloud	Own	One of the oldest real-time services (founded 2010, same year as Pusher). Strong presence features, 15+ years of reliability. Good for IoT and large-scale pub/sub.

Laravel Reverb	Self-hosted (free)	Pusher	Official Laravel WebSocket server. You run it alongside your Laravel app. Free, no external dependency, tight Laravel integration. Used in this project.
Soketi	Self-hosted (free)	Pusher	Open-source Pusher-compatible server written in Node.js. A predecessor to Reverb in the Laravel ecosystem. Lighter weight, but less actively maintained now that Reverb exists.
Laravel WebSockets	Self-hosted (free)	Pusher	A PHP package (by BeyondCode) that added WebSocket support to Laravel before Reverb was official. Now largely superseded by Reverb.
Socket.IO	Self-hosted (free)	Own (Socket.IO)	Node.js library with its own protocol on top of WebSockets. Has automatic fallback to long polling, rooms, namespaces. Very popular in the Node ecosystem. Not Pusher-compatible — requires its own client library.
gRPC streaming	Protocol (not a service)	HTTP/2 + Protobuf	Not a real-time pub/sub system — it is a high-performance RPC framework with streaming support. Used for service-to-service communication, not browser/mobile real-time features. Requires Protobuf schema definitions.

6. What is the difference between WebSockets and Pusher?

They operate at different levels:

- **WebSocket** is a *low-level network protocol* (RFC 6455). It establishes a persistent, bidirectional TCP connection between client and server. It only gives you raw message sending and receiving — no concept of channels, rooms, auth, or events. You build everything yourself.
- **Pusher protocol** is a *higher-level application protocol built on top of WebSockets*. It adds structure: named channels, event names, presence member lists, channel authentication, and a defined message format (JSON). It solves the problems you would otherwise have to solve yourself.

```
WebSocket (raw)
└── you send: "hello"
└── you receive: "world"
    (no structure, no channels, no auth)

Pusher protocol (on top of WebSocket)
└── subscribe to channel "presence-room.ABC"
└── bind to event "message.sent"
└── receive: { user: "John", content: "hello" }
    (structured, authenticated, channel-isolated)
```

Think of WebSocket as a phone line and Pusher protocol as the language and rules you agree to speak over that line.

7. Why are events needed in Pusher?

A single channel can carry many different *types* of data. Without events, you would receive a raw stream and have no way to distinguish what each message means.

Events give each message a **name**, so the client can route it to the right handler:

```
// Without events — raw WebSocket
onMessage: (data) {
  // Is this a chat message? A user joined? A typing indicator?
  // You have to parse and guess.
}

// With Pusher events
channel.bind('message.sent',    (data) => showMessage(data));
channel.bind('user.typing',     (data) => showTypingIndicator(data));
channel.bind('user.joined',     (data) => updateMemberList(data));
channel.bind('room.closed',     (data) => navigateBack());
```

Events also let multiple parts of the UI subscribe to the *same channel* but listen for *different events* independently — without any shared state or routing logic. They are the core organisational unit of the Pusher protocol.